

# Analyzing a Wireshark Packet Capture File

**Summary:** In this lab, I used Wireshark to analyze a packet capture file and investigate network traffic associated with a user connecting to a website. My role was to act as a security analyst examining packet data to identify IP addresses, protocols, and payload information relevant to a web browsing session. Practicing how to apply Wireshark filters and explore packet layers is essential for understanding how systems communicate across a network.

## Wireshark Filters:

```
ip.addr == x.x.x.x
ip.src == x.x.x.x
ip.dst == x.x.x.x
eth.addr == xx:xx:xx:xx:xx:xx
udp.port == xx
tcp.port == xx
tcp contains "text"
```

**Overview:** To begin the investigation, I opened a packet capture file in Wireshark to explore how network traffic is displayed. Wireshark organizes packet data into columns such as source, destination, protocol, and packet length. Each of these columns helps analysts quickly scan large amounts of traffic more efficiently. I explored the three-pane interface, which includes the packet list, packet details, and raw hexadecimal data.

Next, I applied basic display filters to narrow down targeted packets. Using filters like `ip.addr ==` allowed me to isolate packets associated with a specific IP address. This helped reduce noise and focus only on communication between two systems. Using `ip.src` and `ip.dst` can refine the results even further by grabbing just the source or destination IP. I then inspected individual packets by expanding protocol layers such as Frame, Ethernet, IPv4, and TCP. Each layer reveals different information, such as MAC or IP addresses, ports, and protocol-specific details.

After reviewing the IP based traffic, I moved on to filtering DNS packets by using `udp.port == 53`. This allowed me to examine DNS queries and responses, including the domain names being requested and the IP addresses returned by the DNS server. Inspecting the DNS layer helped me understand how name resolution occurs during a web browsing session and how Wireshark displays both the query and the corresponding answer.

Finally, I analyzed TCP traffic by filtering for packets on port 80, which is commonly used for web traffic. This made it possible to inspect details such as TTL values, frame lengths, and header sizes within individual packets. I also used the TCP contains filter to search for specific text within packet payloads, which is useful when identifying commands, user-agent strings, or other meaningful data transmitted during a session. These filtering techniques demonstrated how Wireshark can isolate both protocol-level behavior and the actual content being exchanged between systems.

**Reflection:** This lab reinforced how powerful Wireshark is for analyzing activity during an investigation. Using filters made it easy to cut through large amounts of packet data and focus on the traffic that mattered. Inspecting protocol layers helped me understand how data moves across a network and how different protocols interact. These skills are essential for identifying suspicious behavior, tracing communication paths, and validating whether traffic is legitimate or potentially malicious. Overall, this lab strengthened my ability to interpret packet data and apply targeted filters to support real-world security investigations.